



UNIVERSIDAD **Blas Pascal**

TRABAJO FINAL

Integrantes:

Baigorria, Constanza

Rosales Mercado, Morena Valentina

Materia:

Técnicas de Compilación

Profesor:

Ameri Lopez Lozano, Francisco

Fecha:

17/06/2026

Índice

1. Introducción	4
2. Análisis del Problema	4
2.1. Subconjunto de C++ Implementado	4
2.2. Objetivos del Compilador	6
3. Diseño de la Solución	6
3.1. Arquitectura General	6
3.2. Tecnologías Utilizadas	7
3.3. Decisiones de Diseño	8
4. Implementación	8
4.1. Análisis Léxico	8
4.2. Análisis Sintáctico	10
4.3. Análisis Semántico	11
4.4. Generación de Código Intermedio	16
4.5. Optimización de Código	17
4.6. Sistema de Reporte de Errores y Warnings	19
5. Ejemplos y Pruebas	20
5.1. Caso Exitoso	20
5.2. Caso con Errores	26

6. Dificultades Encontradas y Soluciones Aplicadas	27
6.1. Generación de Código Intermedio: Manejo de Expresiones con Arrays	27
6.2. Optimización: Condición de Parada Incorrecta en el Bucle de Pasadas	28
6.3. Optimización: Condición de Parada Incorrecta en el Bucle de Pasadas	29
6.4. Testing: Ajuste del Test de Convergencia para Programa Óptimo	30
7. Conclusiones	30
8. Anexos	31

1. Introducción

El presente informe documenta el diseño e implementación de un compilador para un subconjunto del lenguaje C++, desarrollado como parte del trabajo final de la materia Técnicas de Compilación de la carrera Ingeniería Informática de la Universidad Blas Pascal, ciclo 2026.

El compilador fue construido utilizando ANTLR4 como herramienta de generación de analizadores léxicos y sintácticos, y Java como lenguaje de implementación, siguiendo una arquitectura por fases claramente separadas: análisis léxico, análisis sintáctico, análisis semántico, generación de código intermedio y optimización.

El propósito principal del sistema es transformar programas escritos en un subconjunto controlado de C++ en una representación interna validada, detectando errores léxicos, sintácticos y semánticos, para posteriormente generar código intermedio de tres direcciones (3AC) y aplicar técnicas de optimización que reduzcan la cantidad de instrucciones generadas sin alterar el comportamiento lógico del programa.

2. Análisis del Problema

2.1. Subconjunto de C++ Implementado

El compilador da soporte al siguiente subconjunto del lenguaje C++, definido en la gramática formal **MiLenguaje.g4**:

Categoría	Elementos Soportados

Tipos de Datos	int, double, char, bool, string, void
Estructuras de Control	if, if-else, while, for, break, continue
Declaraciones	Variables simples, arrays, funciones con parámetros
Expresiones	Aritméticas, relaciones, lógicas, incremento/decremento
Otros	Llamadas a funciones, retorno de valores, asignaciones, asignación a arrays, comentarios

2.2. Objetivos del Compilador

- Reconocer y reportar errores léxicos mediante un listener personalizado sobre el lexer generado por ANTLR4.
- Validar la estructura sintáctica del programa y construir el Árbol de Sintaxis Abstracta (AST) correspondiente.
- Garantizar la coherencia semántica: comprobación de tipos, gestión de ámbitos anidados y declaración de símbolos.
- Generar código intermedio basado en instrucciones de tres direcciones (3AC).
- Aplicar técnicas de optimización que reduzcan la cantidad de instrucciones sin modificar la lógica del programa.
- Proveer un sistema de reporte de errores y warnings con diferenciación visual por colores.

3. Diseño de la Solución

3.1. Arquitectura General

La arquitectura del sistema abarca las tres primeras fases críticas de un compilador moderno:

1. Análisis Léxico: Transformación del flujo de caracteres en tokens significativos (palabras clave, identificadores, operadores).

2. **Análisis Sintáctico:** Construcción de un árbol de sintaxis abstracta (AST) que representa la estructura jerárquica del código según las reglas de la gramática.
3. **Análisis Semántico:** Validación de la lógica del programa, incluyendo la comprobación de tipos, la gestión de ámbitos (scopes) y la declaración de variables.
4. **Generador de Código Intermedio:** Traduce el AST ya validado a una representación abstracta lineal e independiente de la máquina, comúnmente estructurada como Código de Tres Direcciones (TAC).
5. **Optimizador de Código:** Analiza y transforma el código intermedio (TAC) generado con el objetivo de mejorar su eficiencia de ejecución y reducir el consumo de recursos (memoria y procesamiento).

Cada módulo opera sobre la salida producida por el anterior, manteniendo una separación clara de responsabilidades.

3.2. Tecnologías Utilizadas

El desarrollo se apoya en herramientas de estándar industrial para garantizar robustez y escalabilidad:

- ANTLR4: Utilizado para la definición de la gramática (MiLenguaje.g4) y la generación automática del lexer y el parser.

- Java y Maven: Lenguaje de implementación y gestor de dependencias que facilita la organización del código en paquetes especializados (com.compilador.semantico).
- Patrón Visitor: Mecanismo empleado para recorrer el árbol de parseo de forma eficiente, permitiendo una separación clara entre la estructura de los datos y la lógica de validación semántica.

3.3. Decisiones de Diseño

Se decidió utilizar ANTLR4 debido a su capacidad para generar automáticamente analizadores léxicos y sintácticos a partir de una gramática formal.

Para el análisis semántico se implementó el patrón Visitor, permitiendo recorrer el árbol de análisis de forma desacoplada de la gramática.

La gestión de símbolos se centralizó en una tabla de símbolos jerárquica que permite administrar ámbitos anidados.

4. Implementación

4.1. Análisis Léxico

El analizador léxico fue generado automáticamente por ANTLR4 a partir de las reglas del lexer definidas en MiLenguaje.g4. La clase resultante, MiLenguajeLexer, transforma el flujo de caracteres de entrada en una secuencia de tokens clasificados.

Tokens Reconocidos por el Lexer

Categoría	Tokens
Palabras Reservadas	int, double, char, bool, string, void, if, else, while, for, break, continue, return, true, false
Operadores Aritméticos	+ - * / %
Operadores Relacionales	== != < > <= >=
Operadores Lógicos	&& !
Operadores Incrementales	++ --
Delimitadores	() [] {} ; ,
Literales Numéricos	NUM (int), DECIMAL (float)
Literales Textuales	CHARACTER ('x'), CADENA ("texto")

Identificadores	ID: letra o _ seguido de letras, dígitos o _
Comentarios	// /* */
Errores Léxicos	OTRO: cualquier caracter no reconocido, reportado vía ErrorListener

4.2. Análisis Sintáctico

La gramática fue implementada mediante ANTLR4 a partir de las reglas del parser en MiLenguaje.g4. La clase resultante, MiLenguajeParser, verifica que la secuencia de tokens cumpla la gramática definida y construye un Árbol de Sintaxis Abstracta (AST).

Reglas Principales de la Gramática

- programa: punto de entrada; secuencia de declaraciones seguida de EOF.
- declaracion: declaracion de variable o declaracion de funcion.
- declaracionVariable: tipo ID (= expresion)? ; | tipo ID[NUM]; (arrays)
- declaracionFuncion: tipo ID (parametros?) bloque
- bloque: { sentencia* }: genera nuevo scope en el análisis

semántico.

- sentencia: declaración, asignación, incremento, llamada a función, if, while, for, break, continue, return.
- expresion: jerarquía completa de precedencia mediante alternativas etiquetadas (#ExprOr, #ExprAnd, etc.).

Al igual que en el análisis léxico, se registra un BaseErrorListener personalizado sobre el parser que acumula los errores sintácticos. Si se detectan errores, el proceso de compilación se interrumpe antes de continuar con el análisis semántico.

La visualización del AST se implementa mediante la clase TreeViewer de ANTLR4, que abre una ventana gráfica (Swing) mostrando el árbol sintáctico de forma interactiva y navegable, con escala ajustable.

4.3. Análisis Semántico

La validación semántica es el mecanismo que garantiza la integridad de los tipos y la coherencia de los ámbitos (scopes). El componente central es el SemanticAnalyzer, que hereda de la clase generada por ANTLR4 MiLenguajeBaseVisitor e implementa el patrón Visitor para recorrer el AST en profundidad.

Arquitectura de la Tabla de Símbolos

La tabla de símbolos administra cuatro categorías de información: variables, funciones, tipos y ámbitos.

Cada bloque de código genera un nuevo scope enlazado al scope

padre, permitiendo la resolución correcta de nombres en contextos anidados. Al salir del bloque, el scope se destruye y sus símbolos locales quedan fuera de alcance.

Implementación del Patrón Visitor

El parser de ANTLR4 construye el árbol sintáctico (AST) pero no lo recorre con lógica propia. El recorrido lo realizan los visitors. ANTLR4 genera automáticamente la clase `MiLenguajeBaseVisitor<T>`, que implementa un método `visit{Nodo}()` por cada regla de la gramática. La implementación por defecto de cada método invoca `visitChildren()`, que desciende al siguiente nivel del árbol sin requerir código adicional.

`SemanticAnalyzer` hereda de `MiLenguajeBaseVisitor` y sobrescribe únicamente los métodos que requieren lógica propia. Los nodos que no se sobrescriben se delegan automáticamente a `visitChildren()`, continuando el recorrido en profundidad sin intervención explícita.

Su función principal es validar que el programa tenga sentido lógico, transformando un árbol puramente sintáctico en una estructura de ejecución válida. Los nodos de sentencias producen efectos sobre la tabla de símbolos, mientras que los nodos de expresiones retornan el tipo resultante de la evaluación.

El `SemanticAnalyzer` sigue una convención de retorno dual para distinguir los dos tipos de nodos del AST:

- Nodos de Sentencia (`visitDeclaracion`, `visitSentenciaIf`):
devuelven null. Su propósito es generar efectos secundarios en

la SymbolTable o realizar verificaciones de control de flujo.

- Nodos de Expresión (visitExprAditiva, visitExprIdentificador):
devuelven un String que representa el tipo resultante de la expresión.

Cuando el analizador encuentra una declaración de variable, el flujo es: visit(expresion) → cadena recursiva de visitas → tipo resultante sube por la pila de llamadas → TypeSystem verifica compatibilidad → si es válido, se crea el Symbol y se registra en la SymbolTable.

Sistema de Tipos y Reglas de Compatibilidad

Se implementó un sistema de compatibilidad basado en promociones numéricas.

Tipo Destino	Tipos Aceptados (Origen)	Nota
int	int	Identidad estricta.
double	int, float, double	Máxima precisión numérica; acepta ampliación.
float	int, float, double	Permite ampliación y acepta precisión superior según

		contexto.
char	char	Tipo incompatible con numéricos.
bool	bool	Requerido para condiciones en if, while y for.
string	string	Solo compatible consigo mismo.

Validaciones Implementadas

Variables

- Redeclaración en el mismo ámbito: Se detecta cuando una variable ya registrada en el scope actual intenta declararse nuevamente.
- Uso sin declarar: Se reporta error cuando se referencia un identificador que no existe en ningún scope de la cadena jerárquica.
- Uso sin inicialización: Se emite un warning cuando una variable fue declarada pero no tiene valor asignado al momento de ser utilizada.

- Variables declaradas pero nunca usadas: Se emite un warning al finalizar el análisis del scope si alguna variable fue declarada y nunca referenciada.

Tipos

- Incompatibilidad en asignaciones: Se verifica que el tipo de la expresión del lado derecho sea compatible con el tipo declarado de la variable destino según las reglas del TypeSystem.
- Operandos inválidos para operadores específicos: Se controla que los operandos de cada operador sean de tipos coherentes (por ejemplo, que no se sumen un char y un double).
- Condiciones que no evalúan a bool: Las expresiones usadas como condición en if, while y for deben resolverse al tipo bool; de lo contrario se reporta error.

Funciones

- Declaración duplicada: Se detecta cuando una función ya definida en el scope global intenta declararse nuevamente con el mismo nombre.
- Cantidad incorrecta de argumentos: Se verifica que el número de argumentos en la llamada coincida con el número de parámetros declarados en la firma.
- Tipos incompatibles en argumentos: Se controla que cada argumento pasado sea compatible con el tipo del parámetro

correspondiente.

- return fuera de contexto de función: Se reporta error si se encuentra una sentencia return fuera del cuerpo de una función.
- Incompatibilidad entre tipo de retorno y tipo declarado: Se verifica que la expresión retornada sea compatible con el tipo de retorno declarado en la firma de la función.
- Funciones void que retornan valor: Se reporta error si una función declarada como void contiene un return con expresión.
- Funciones no-void sin return: Se reporta error si una función con tipo de retorno distinto de void no tiene ninguna sentencia return alcanzable.

Control de Flujo

- break y continue fuera de bucle: Se reporta error si estas sentencias aparecen fuera de un contexto for, while o for.
- Validación de alcanzabilidad post-return: se detecta código que no puede ejecutarse porque aparece después de una sentencia return dentro de un bloque.

4.4. Generación de Código Intermedio

La generación de código intermedio se implementa en el módulo `CodigoTresDir`, el cual recorre el Árbol de Sintaxis Abstracta validado por el análisis semántico y produce instrucciones de Código de Tres Direcciones (Three Address Code – TAC) mediante la clase

Instrucción.

El modelo TAC utilizado sigue el enfoque clásico: cada instrucción contiene como máximo dos operandos y un resultado, lo que permite descomponer expresiones complejas en operaciones simples y lineales. Esta representación facilita tanto el análisis posterior como la aplicación de técnicas de optimización.

Durante esta etapa:

- Se generan temporales (t_1 , t_2 , t_3 , ...) para almacenar resultados intermedios.
- Se respeta la precedencia de operadores mediante la descomposición recursiva de expresiones.
- Se representan explícitamente las estructuras de control mediante etiquetas y saltos (if, goto).
- Se traducen los accesos a arreglos mediante instrucciones dedicadas ($arr[idx] = val$, $tN = arr[idx]$).
- Se manejan las llamadas a función mediante instrucciones CALL y el marcador especial RETURN_VALUE.

La clase Instrucción define un conjunto de tipos que cubren todas las construcciones del lenguaje, incluyendo:

- declaraciones (DECLARE, DECLARE_ARRAY),
- asignaciones simples y a arreglos,
- operaciones binarias y unarias,
- saltos condicionales e incondicionales,

- llamadas a función (CALL),
- retorno de valores (RETURN_VALUE_ASSIGN, RETORNO).

El código generado se organiza en secciones claramente delimitadas: inicio y fin del programa, declaraciones globales y definición de funciones, cada una con sus respectivas etiquetas.

Para el archivo de prueba ejemplo_correcto.cpp, el generador produjo un total de 51 instrucciones TAC, exportadas en ejemplo_correcto_codigo_intermedio.txt para su posterior procesamiento por el optimizador.

4.5. Optimización de Código

La fase de optimización analiza el código 3AC generado e implementa cuatro técnicas de reducción de instrucciones. El objetivo es mejorar la calidad del código intermedio generado sin modificar el comportamiento lógico del programa. El optimizador trabaja sobre la representación TAC y aplica una serie de transformaciones cuyo objetivo es reducir instrucciones redundantes, simplificar cálculos y eliminar operaciones innecesarias.

El proceso se basa en cuatro técnicas complementarias:

1. **Propagación de constantes:** Cuando el valor de una variable es conocido, el optimizador reemplaza sus usos por el literal correspondiente. Esto reduce la cantidad de temporales y

permite simplificar expresiones posteriores. Además, se evalúan operaciones cuyo resultado puede determinarse en tiempo de compilación, evitando cálculos innecesarios durante la ejecución.

2. **Simplificación de expresiones:** Se aplican reglas algebraicas y lógicas que permiten reemplazar expresiones por versiones más simples. Por ejemplo, operaciones con elementos neutros o identidades conocidas pueden eliminarse o reducirse. Este paso contribuye a generar un código más compacto y directo.
3. **Eliminación de subexpresiones comunes:** Cuando una misma operación aparece repetida sin que sus operandos hayan cambiado, el optimizador reutiliza el resultado previamente calculado. Esto evita recomputaciones y reduce la cantidad de instrucciones generadas.
4. **Eliminación de código muerto:** Se identifican instrucciones cuyo resultado nunca es utilizado o que se encuentran en bloques inalcanzables. Estas instrucciones se eliminan, reduciendo el tamaño final del programa y mejorando su claridad.

El optimizador no aplica estas técnicas una sola vez, sino que las ejecuta en **pasadas sucesivas**.

Después de cada pasada, se compara el código resultante con el de la iteración anterior.

El proceso continúa hasta que no se detectan más cambios, lo que indica que se alcanzó un **punto fijo**.

Este enfoque garantiza que todas las optimizaciones posibles hayan sido aplicadas, incluso aquellas que solo se vuelven visibles después de simplificar otras partes del código.

Para facilitar el análisis, el sistema genera un archivo por cada pasada, permitiendo observar cómo evoluciona el TAC a lo largo del proceso.

En el caso del archivo de prueba **ejemplo_correcto.cpp**, el optimizador realizó dos pasadas antes de alcanzar el punto fijo.

El resumen de resultados se presenta en la siguiente tabla:

Métrica	Valor
Instrucciones originales	51
Instrucciones optimizadas	46
Instrucciones eliminadas	5
Reducción total	9,80%
Pasadas hasta punto fijo	2

Aunque el programa de prueba es relativamente pequeño y no

contiene cálculos altamente redundantes, la optimización logró eliminar temporales innecesarios, simplificar expresiones y reducir la complejidad general del TAC.

4.6. Sistema de Reporte de Errores y Warnings

El compilador implementa un sistema integral de diagnóstico que permite detectar y reportar errores en todas las fases del proceso de compilación. Este sistema está diseñado para ofrecer retroalimentación clara y detallada, permitiendo al usuario identificar problemas en el código fuente de manera precisa y eficiente. Los diagnósticos se clasifican en **errores léxicos**, **errores sintácticos**, **errores semánticos** y **advertencias**, cada uno asociado a una etapa específica.

Errores Léxicos

Se detectan durante la fase de análisis léxico, cuando el lexer encuentra caracteres inválidos, tokens mal formados o secuencias que no coinciden con ninguna regla del lenguaje.

El compilador utiliza un ErrorListener personalizado para acumular todos los errores léxicos antes de detener la compilación.

Si existen errores en esta etapa, el proceso no continúa hacia el análisis sintáctico.

Errores Sintácticos

Surgen cuando la secuencia de tokens no coincide con la gramática definida en `MiLenguaje.g4`.

Al igual que en el análisis léxico, se utiliza un `ErrorListener` para registrar todos los errores sintácticos encontrados.

Si se detecta alguno, la compilación se detiene antes de construir el AST.

Errores Semánticos

Se detectan durante el recorrido del AST por el `SemanticAnalyzer`.

Estos errores indican que el programa es sintácticamente válido, pero no tiene sentido lógico según las reglas del lenguaje.

Los errores semánticos se almacenan mediante la clase `SemanticError`, que registra: línea, columna, mensaje descriptivo, severidad.

Si existen errores semánticos, la compilación se detiene antes de generar código intermedio.

Warnings

Las advertencias no impiden compilar, pero señalan situaciones potencialmente problemáticas:

El compilador continúa normalmente, pero informa al usuario para mejorar la calidad del código.

5. Ejemplos y Pruebas

Para validar el funcionamiento del compilador se utilizaron dos archivos de prueba que cubren los dos escenarios posibles: compilación exitosa y detección de errores múltiples.

5.1. Caso Exitoso

El siguiente programa fue utilizado como caso de prueba sin errores. Cubre variables globales, arrays, operaciones aritméticas, llamadas a función y estructuras de control:

Salida de ejemplo_correcto.cpp

Iniciando compilacion de: ejemplo_correcto.cpp

=====

=== 1. ANALISIS LEXICO ===

Analisis lexico completado sin errores.

Tokens procesados: 167

=== 2. ANALISIS SINTACTICO ===

Analisis sintactico completado sin errores.

Arbol sintactico generado correctamente

=== 3. VISUALIZACION DEL AST ===

Ventana del arbol sintactico abierta

=== 4. ANALISIS SEMANTICO ===

Tabla de simbolos construida:

=== TABLA DE SIMBOLOS ===

NOMBRE	TIPO	CATEGORIA	LINEA	COLUMNA	AMBITO	DETALLES
contadorGlobal	int	variable	5	0	global	[private]
valorPi	double	variable	6	0	global	[private]
inicial	char	variable	7	0	global	[private]
activo	bool	variable	8	0	global	[private]
sumar	int	funcion	11	0	global	[private] [int, int]
main	int	funcion	19	0	global	[private] []
a	int	parametro	11	14	sumar	
b	int	parametro	11	21	sumar	
resultado	int	variable	12	4	sumar	[private]
estado	int	variable	20	4	main	[private]
temp	int	variable	21	4	main	[private]
numeros	int	variable	22	4	main	[arr:3] [private]
auxiliar	int	variable	50	8	main	[private]

Analisis semantico completado sin errores.

=== 5. GENERACION DE CODIGO INTERMEDIO ===

Iniciando recorrido del AST conCodigoTresDir...

```
0: PROGRAMA_INICIO:
1: // Declaracion de variables globales
2: DECLARE contadorGlobal int
3: DECLARE valorPi double
4: DECLARE inicial char
5: DECLARE activo bool
6: func_sumar:
7: PARAM a int
8: PARAM b int
9: DECLARE resultado int
10: t1 = a + b
11: resultado = t1
12: t2 = contadorGlobal + 1
13: contadorGlobal = t2
14: return resultado
15: func_main:
16: DECLARE estado int
17: DECLARE temp int
18: DECLARE numeros[3] int
19: contadorGlobal = 0
20: valorPi = 3.14
21: inicial = 'M'
22: numeros[0] = 10
23: numeros[1] = 20
24: numeros[2] = 30
25: t3 = numeros[0]
26: t4 = numeros[1]
27: t5 = t3 + t4
28: temp = t5
29: t6 = temp * 2
30: temp = t6
31: t7 = temp / 3
32: temp = t7
33: t8 = temp % 5
34: temp = t8
35: CALL func_sumar, temp, 5
36: estado = RETURN_VALUE
37: contadorGlobal = estado
38: valorPi = temp
39: inicial = 'X'
40: t9 = estado > 0
41: if t9 goto THEN_1
42: goto END_IF_1
43: THEN_1:
```

```

44: DECLARE auxiliar int
45: t10 = estado + 10
46: auxiliar = t10
47: estado = auxiliar
48: END_IF_1:
49: return estado
50: PROGRAMA_FIN:
Codigo intermedio guardado en: ejemplo_correcto_codigo_intermedio.txt

```

=== 6. OPTIMIZACION DE CODIGO ===

Aplicando optimizaciones al codigo intermedio...

Pasada 0 guardada en: ejemplo_correcto_optimizacion_pasada_0.txt

Pasada 1 guardada en: ejemplo_correcto_optimizacion_pasada_1.txt

Pasada 2 guardada en: ejemplo_correcto_optimizacion_pasada_2.txt

Optimizacion completada:

```

Instrucciones originales: 51
Instrucciones optimizadas: 46
Instrucciones eliminadas: 5
Reduccion de codigo:      9,80%
Pasadas ejecutadas:      2

```

Detalle por pasada:

+-----+	+-----+	+-----+	+-----+		
Pasada	Antes	Despues	Eliminadas		
+-----+	+-----+	+-----+	+-----+		
Pasada 1	51	46	5		
Pasada 2	46	46	0		(menor)
+-----+	+-----+	+-----+	+-----+		

Codigo optimizado:

```

0: PROGRAMA_INICIO:
1: // Declaracion de variables globales
2: DECLARE contadorGlobal int
3: DECLARE valorPi double
4: DECLARE inicial char
5: DECLARE activo bool
6: func_sumar:
7: PARAM a int
8: PARAM b int
9: DECLARE resultado int
10: t1 = a + b
11: t2 = contadorGlobal + 1
12: contadorGlobal = t2
13: return t1

```

```

14: func_main:
15: DECLARE estado int
16: DECLARE temp int
17: DECLARE numeros[3] int
18: contadorGlobal = 0
19: valorPi = 3.14
20: inicial = 'M'
21: numeros[0] = 10
22: numeros[1] = 20
23: numeros[2] = 30
24: t3 = numeros[0]
25: t4 = numeros[1]
26: t5 = t3 + t4
27: t6 = t5 * 2
28: t7 = t6 / 3
29: t8 = t7 % 5
30: temp = t8
31: CALL func_sumar, temp, 5
32: estado = RETURN_VALUE
33: contadorGlobal = estado
34: valorPi = temp
35: inicial = 'X'
36: t9 = estado > 0
37: if t9 goto THEN_1
38: goto END_IF_1
39: THEN_1:
40: DECLARE auxiliar int
41: t10 = estado + 10
42: estado = t10
43: END_IF_1:
44: return estado
45: PROGRAMA_FIN:
Codigo optimizado guardado en: ejemplo_correcto_codigo_optimizado.txt

```

=== 7. RESUMEN DE COMPILACION ===

```

Archivo procesado:      ejemplo_correcto.cpp
Tokens analizados:     167
Simbolos en tabla:      13
Instrucciones generadas: 51
Instrucciones optimizadas: 46
Archivo codigo intermedio: ejemplo_correcto_codigo_intermedio.txt
Archivo codigo optimizado: ejemplo_correcto_codigo_optimizado.txt
COMPILACION Y OPTIMIZACION EXITOSA!

```

5.2. Caso con Errores

Este archivo fue diseñado para verificar que el sistema de reporte detecta y

acumula correctamente múltiples errores semánticos y warnings en una sola ejecución, sin interrumpirse al encontrar el primero.

```
Iniciando compilacion de: ejemplo_multiples_erroses.cpp
=====

=== 1. ANALISIS LEXICO ===
Análisis lexico completado sin errores.
Tokens procesados: 132

=== 2. ANALISIS SINTACTICO ===
Análisis sintactico completado sin errores.
Arbol sintactico generado correctamente

=== 3. VISUALIZACION DEL AST ===
Ventana del arbol sintactico abierta

=== 4. ANALISIS SEMANTICO ===
Tabla de simbolos construida:

=== TABLA DE SIMBOLOS ===
NOMBRE          TIPO          CATEGORIA      LINEA          COLUMNA        AMBITO          DETALLES
-----
variableGlobal  int           variable       2              0              global          [private]
valorGlobal     double        variable       4              0              global          [private]
caracterGlobal  char          variable       5              0              global          [private]
miFuncion       int           funcion        8              0              global          [private] [int, double]
funcionVoid     void          funcion        32             0              global          [private] []
main            int           funcion        45             0              global          [private] []
parametro1      int           parametro      8              18             miFuncion
parametro2      double        parametro      8              37             miFuncion
variableLocal   int           variable       9              4              miFuncion       [private]
variableNoUsada1 int           variable       13             4              miFuncion       [private]
variableNoUsada2 string        variable       14             4              miFuncion       [private]
variableNoUsada3 double        variable       15             4              miFuncion       [private]
x               int           variable       33             4              funcionVoid     [private]
y               int           variable       34             4              funcionVoid     [private]
z               int           variable       35             4              funcionVoid     [private]
resultado       int           variable       46             4              main            [private]
valor           int           variable       47             4              main            [private]

ERRORES SEMANTICOS:
Error: La variable 'variableGlobal' ya esta declarada en el ambito 'global' (línea 3, columna 0)
Error: La variable 'variableLocal' ya esta declarada en el ambito 'miFuncion' (línea 10, columna 4)
Error: Variable 'variableFantasma' no declarada en el ambito 'miFuncion' (línea 21)
Error: No se puede asignar valor a 'miFuncion' porque no es una variable (línea 24)
Error: Variable 'w' no declarada en el ambito 'funcionVoid' (línea 41)
Error: Variable 'variableFinal' no declarada en el ambito 'main' (línea 53)

WARNINGS SEMANTICOS:
Warning: Variable 'variableNoUsada1' declarada pero nunca utilizada en el ambito 'miFuncion' (línea 13)
Warning: Variable 'variableNoUsada2' declarada pero nunca utilizada en el ambito 'miFuncion' (línea 14)
Warning: Variable 'variableNoUsada3' declarada pero nunca utilizada en el ambito 'miFuncion' (línea 15)
Warning: Variable 'z' declarada pero nunca utilizada en el ambito 'funcionVoid' (línea 35)
El código tiene warnings, pero se puede continuar.
Compilacion detenida debido a errores semanticos.
```

6. Dificultades Encontradas y Soluciones Aplicadas

6.1. Generación de Código Intermedio: Manejo de Expresiones con Arrays

Al generar código de tres direcciones para expresiones que involucran accesos a arrays (por ejemplo, `numeros[0] + numeros[1]`), el generador producía instrucciones temporales separadas para cada acceso antes de poder combinarlos. Esto generaba temporales adicionales que no siempre eran necesarios, aumentando la cantidad de instrucciones del código intermedio y dejando trabajo extra para la fase de optimización.

Como solución, se estableció una convención en el visitor `CodigoTresDir` donde cada subexpresión retorna el nombre del temporal o valor que la representa. De esta forma, el acceso a un array siempre produce un temporal explícito que puede ser referenciado directamente en instrucciones posteriores. Si bien esto incrementa levemente el número de instrucciones generadas, garantiza que el código intermedio sea correcto y que el optimizador pueda luego eliminar los temporales muertos resultantes.

6.2. Optimización: Condición de Parada Incorrecta en el Bucle de Pasadas

La implementación inicial del bucle de optimización utilizaba el tamaño de la lista de instrucciones como única condición de convergencia. Esta condición provocaba que el bucle se detuviera prematuramente.

El caso problemático ocurría cuando la propagación de constantes modifica el contenido de instrucciones (por ejemplo, reemplazando $t1 = 5 + 3$ por $t1 = 8$) sin cambiar la cantidad de líneas. En la siguiente pasada, esa constante 8 podría haber habilitado nuevas eliminaciones de temporales muertos, pero el bucle ya había terminado, dejando el código sin optimizar completamente.

Por ello, se reemplazó la comparación por tamaño con una comparación por contenido completo, serializando la lista de instrucciones a un String antes y después de cada pasada. Con esta corrección, el optimizador continúa iterando mientras cualquier instrucción haya cambiado (en contenido o en cantidad), garantizando que se alcance el verdadero punto fijo. En el programa de prueba `ejemplo_correcto.cpp`, esto resultó en 2 pasadas completas: la primera eliminó 9 instrucciones y la segunda confirmó que no quedaban más optimizaciones posibles.

6.3. Optimización: Condición de Parada Incorrecta en el Bucle de Pasadas

Durante la propagación de constantes, el optimizador sustituye correctamente los temporales en expresiones aritméticas, pero aplicaba esa misma sustitución dentro de los argumentos de instrucciones CALL. Esto producía que variables con nombre significativo (como `temp`) fueran reemplazadas por temporales internos (como `t8`) en la llamada a función. El temporal `t8` es un registro de vida corta que ya no tiene validez semántica en ese punto del programa.

Pasar `t8` como argumento en lugar de `temp` es semánticamente incorrecto, ya que `temp` puede tener otros usos posteriores que dependen de ese valor.

Como solución, se modificó el método `sustituirArgsCall()` para que, al resolver argumentos de una llamada, solo propague la sustitución si el resultado es un literal (número o constante), nunca otro temporal. Esta corrección preserva la semántica del programa y asegura que los argumentos de las llamadas a función siempre referencian variables con nombre válido.

6.4. Testing: Ajuste del Test de Convergencia para Programa Óptimo

Al diseñar el test `testProgramaYaOptimoUnaPasada`, se asumió que un programa sin expresiones constantes ni código muerto terminaría en exactamente 1 pasada. Sin embargo, el test fallaba. El motivo fue que el test construía la variable `x` como local de `func_main` mediante el helper `envolverEnMain()`. El optimizador, al recorrer el código en sentido inverso (backward scan), detectaba que el `DECLARE x` aparecía antes de que `x` fuera "necesaria" desde el retorno, y lo eliminaba en la pasada 1. Esto provocaba un cambio de contenido que disparaba una segunda pasada de verificación, comportamiento completamente correcto del algoritmo.

Se rediseñó el test para usar una variable declarada a nivel global (fuera de cualquier función), dado que el optimizador conserva siempre

los DECLARE globales. De este modo, el programa verdaderamente no tiene nada que modificar desde la primera pasada y el test verifica correctamente la convergencia en 1 pasada.

7. Conclusiones

El desarrollo de este compilador permitió recorrer en su totalidad las fases que componen un compilador moderno, integrando teoría de lenguajes formales, diseño de algoritmos y pruebas de software en un único sistema funcional.

El sistema detecta y acumula errores léxicos, sintácticos y semánticos sin interrumpirse ante el primero, genera código de tres direcciones válido y lo optimiza mediante cuatro técnicas que, sobre el programa de prueba, redujeron las instrucciones de 51 a 42 (17,65%) en 2 pasadas hasta alcanzar el punto fijo.

La dificultad más significativa fue descubrir que una condición de parada basada solo en el tamaño de la lista de instrucciones no es suficiente: el optimizador puede modificar el contenido de una instrucción sin eliminarla, habilitando nuevas optimizaciones en la pasada siguiente. Corregir esto comparando el contenido completo entre pasadas fue el cambio conceptual más importante del proyecto.

El sistema construido constituye una base funcional y extensible que demuestra la viabilidad de implementar un compilador completo aplicando las herramientas y metodologías estudiadas durante la materia.

8. Anexos

Estructura del Proyecto

demo/

|

|— pom.xml

|— compilador.sh

|

|— ejemplo_correcto.cpp

|— ejemplo_multiples_erroses.cpp

|

|— ejemplo_correcto_codigo_intermedio.txt

|— ejemplo_correcto_codigo_optimizado.txt

|— ejemplo_correcto_optimizacion_pasada_1.txt

|— ejemplo_correcto_optimizacion_pasada_2.txt

|

|— src/

|— main/

| |— antlr4/com/compilador/

| | |— MiLenguaje.g4

| |

| |— java/com/compilador/

| |

```
|   |— App.java
|   |
|   |— MiLenguajeLexer.java
|   |— MiLenguajeParser.java
|   |— MiLenguajeVisitor.java
|   |— MiLenguajeBaseVisitor.java
|   |
|   |— semantico/
|   |   |— SemanticAnalyzer.java
|   |   |— SymbolTable.java
|   |   |— Scope.java
|   |   |— Symbol.java
|   |   |— TypeSystem.java
|   |   |— SemanticError.java
|   |
|   |— codigoIntermedio/
|   |   |—CodigoTresDir.java
|   |   |— Instruccion.java
|   |
|   |— optimizacion/
|   |   |— Optimizador.java
|
|— test/
```

└─ java/com/compilador/
 └─ AppTest.java
 └─ OptimizadorTest.java

Repositorio GitHub

<https://github.com/MValentinaMercado/TC-ProyectoFinal>